

Encounters of the Fibrous Kind

Clint Wang, Jeffery Xu

April 30, 2026

1 Introduction and Theory

We built a real-time physical simulation framework for rigid and fabric-like objects using extended position-based dynamics (XPBD) [MMC16]. To evaluate the strengths and limitations of XPBD, we extended a baseline shape-matching PBD cloth simulation with self-collision handling, friction constraints, and collisions against rigid bodies. After discovering the limitations of shape-matching and triangular meshes, we implemented a particle cloth that uses distance constraints. We then created a real-time origami simulation by adding stiff hinge constraints to the cloth model.

1.1 Extended Position-based Dynamics

The baseline cloth simulation uses position-based dynamics (PBD), which can be thought of as a Verlet-like position prediction step, followed by an implicit-style constraint projection. The constraint solver computes a per-constraint position delta:

$$\Delta x = k_j s_j \mathbf{M}^{-1} \nabla C_j(\mathbf{x}_i)$$

The subscript i denotes the iteration index, j denotes the constraint index, $k_j \in [0, 1]$ is the constraint stiffness, C denotes the constraint function, and the scaling factor s_j is derived:

$$s_j = \frac{-C_j(\mathbf{x}_i)}{\nabla C_j \mathbf{M}^{-1} \nabla C_j^T}$$

The issue with PBD is that stiffness depends on the number of constraint projections performed. XPBD solves this by introducing compliance α (block diagonal compliance matrix corresponding to inverse-stiffness) and a Lagrange multiplier per constraint:

$$\Delta \mathbf{x} = \mathbf{M}^{-1} \nabla \mathbf{C}(\mathbf{x}_i)^T \Delta \lambda$$

XPBD solves $\Delta \lambda$ by approximating each constraint as a local linear system and performing a Gauss-Seidel update:

$$\Delta \lambda = \frac{-C_j(\mathbf{x}_i) - \tilde{\alpha}_j \lambda_{ij}}{\nabla C_j \mathbf{M}^{-1} \nabla C_j^T + \tilde{\alpha}_j}$$

XPBD folds the time step into compliance, defining $\tilde{\alpha} = \frac{\alpha}{\Delta t^2}$. Because shape-matching does not easily provide a scalar C , we chose to keep shape-matching in PBD while creating a new distance constraint option in XPBD:

$$C(\mathbf{x}) = |\mathbf{x}_i - \mathbf{x}_j| - L_0$$

Users can toggle between the two.

1.2 Self-Collision and Rigid Bodies

For both the triangle and particle cloth collision-detection, we used a spatial hash. The triangle cloth hashes triangles and runs vertex-triangle continuous collision detection (CCD). The particle cloth hashes particles and runs vertex-vertex and sub-stepping.

Our implementation of continuous-collision detection predicts collisions along a path by solving for the earliest exact t in $\hat{x} = x + dx\Delta t$ where $\hat{x}$ intersects any object. We discovered that CCD with triangles is computationally expensive and assumes linear approximation of motion resulting in tunneling.

To solve this, we implemented a particle cloth simulation where collisions are tested between vertex particles with a set radius. Since XPBD makes stiffness independent of iteration, we can set the number

of iterations at a low number. Macklin et al. state that instead of running 1 timestep and n iterations per frame, they obtained a significant improvement by running n sub-steps and 1 iteration per frame [MSL⁺19]. By decreasing Δt through sub-stepping, we find that tunneling is virtually eliminated without implementing CCD.

We implemented a stable cloth-cloth friction that blends tangential velocities of contacting points:

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + k_{\text{friction}}(\mathbf{v}_{\text{avg}} - \mathbf{v}_i)\Delta t$$

The total average velocity remains conserved and the relative velocity always shrinks.

For rigid bodies, we copied the numerical integrators from the rigid body assignment for the base implementation. We implemented a BVH for complex mesh collision and a signed distance field for simple rigid body collisions. We implemented a cloth-rigid body frictional constraint satisfying Coulomb's law:

$$C_f(\mathbf{x}) = \|(I - \mathbf{nn}^T)(\Delta\mathbf{x}_i - \Delta\mathbf{x}_j)\|, \Delta\lambda = \min(\mu\Delta\lambda_n, \Delta\lambda_f)$$

Where μ is the frictional coefficient and λ_n is the normal Lagrange multiplier.

1.3 GPU

To simulate a high-resolution cloth, we found the CPU to be insufficient. Logically, XPBD fits the GPU SIMD model: compute velocity, constraints, and position updates in parallel. However, distance constraints for stretching and bending are co-dependent. Gauss-Seidel updates the positions themselves:

$$\text{repeat } \mathbf{x}_i \leftarrow \mathbf{x}_i + \Delta\mathbf{x}$$

Each update influences neighboring constraints. An alternative is a Jacobi solve that performs the following:

$$\mathbf{d}_i \leftarrow 0, \text{repeat } \mathbf{d}_i \leftarrow \mathbf{d}_i + \Delta\mathbf{x}, \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{sd}_i$$

However, the Jacobi solve takes longer to converge, overshoots, and violates momentum conservation. Therefore, we adopted a hybrid approach. We used greedy graph coloring to iteratively select and concurrently process a subset of independent distance constraints using Gauss-Seidel. Because graph-coloring would yield a large number of colors (small sets) for collision constraints, we handle collision constraints using a parallel Jacobi solve.

1.4 Origami

The goal of the origami simulation is to map a mesh of creases on an unfolded sheet of paper to the folded version. Typically, the crease mesh is called a crease pattern and will contain 3 types of edges:

1. Boundary edges represent the edge of the paper and typically form a square.
2. Mountain fold edges represent creases that fold away from you.
3. Valley fold edges represent creases that fold towards you.

We base our implementation on the numerical integrator developed by Ghassei et al. [GDG18]. Ghassei et al. model a force $\mathbf{F}_{crease} = -k_{crease}(\theta - \theta_{target})\frac{\partial\theta}{\partial\mathbf{p}}$ based on a dihedral crease constraint, which naturally maps to an XPBD constraint. We evaluate the constraint on hinge diamonds, which are diamonds that overlay creases. In origami, the dihedral angle is more usefully defined as the angle between the two plane normals, rather than between the planes themselves. A flat unfolded hinge diamond will have a dihedral angle of 0 according to our convention. To simulate fold progress, we set $\theta_{target} = \pi c_{\text{progress}}\mathbf{t}$, where $c_{\text{progress}} \in [0, 1]$ and $\mathbf{t}$ is a vector the size of the number of hinges, containing 1 if hinge i is a valley fold, or -1 if hinge i is a mountain fold.

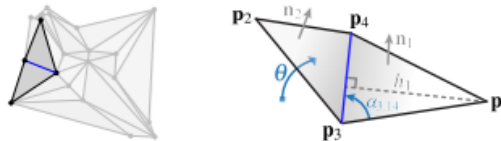


Figure 1: Dihedral crease constraint from Ghassei et al.

The constraint is rather simple:

$$C = \theta - \theta_{goal}$$

We also want to know ∇C_i , the gradient of C at point $\mathbf{x}_i$. We get:

$$\nabla C = \left[\frac{\partial \theta}{\partial \mathbf{x}_1}, \frac{\partial \theta}{\partial \mathbf{x}_2}, \frac{\partial \theta}{\partial \mathbf{x}_3}, \frac{\partial \theta}{\partial \mathbf{x}_4} \right]$$

Following the orientation given by Figure 1, the derivatives for flap vertices $\mathbf{x}_1, \mathbf{x}_2$ are proportional to the normals of their respective triangles:

$$\frac{\partial \theta}{\partial \mathbf{x}_1} = \frac{\mathbf{n}_1}{h_1}, \frac{\partial \theta}{\partial \mathbf{x}_2} = \frac{\mathbf{n}_2}{h_2}$$

The derivatives for edge vertices $\mathbf{x}_3, \mathbf{x}_4$ are the weighted sum of the flap vertex derivatives.

$$\begin{aligned} \frac{\partial \theta}{\partial \mathbf{x}_3} &= \frac{-\cot \alpha_{4,31}}{\cot \alpha_{3,14} + \cot \alpha_{4,31}} \frac{\mathbf{n}_1}{h_1} + \frac{-\cot \alpha_{4,23}}{\cot \alpha_{3,42} + \cot \alpha_{4,23}} \frac{\mathbf{n}_2}{h_2} \\ \frac{\partial \theta}{\partial \mathbf{x}_4} &= \frac{-\cot \alpha_{3,14}}{\cot \alpha_{4,31} + \cot \alpha_{3,14}} \frac{\mathbf{n}_1}{h_1} + \frac{-\cot \alpha_{3,42}}{\cot \alpha_{4,23} + \cot \alpha_{3,42}} \frac{\mathbf{n}_2}{h_2} \end{aligned}$$

2 Implementation Details

We created a fabric simulation harness in Rust that is compiled to Web-Assembly (Wasm), and rendered using WebGPU. The harness can be run in-browser or as a headless binary for advanced debugging. We set up a simple user interface in the browser to adjust the parameters. The github repository can be found at <https://github.com/txst54/wasm-cloth-sim>. A browser-ready version is up at my website: <https://cloth.clintwang.com>.

2.1 Architecture

The Rust core of the simulation is located in `/rust`. All architecture-specific details (Wasm versus native binary) can be found in `/rust/src/arch`. GPU-support is only enabled in the Wasm package, and the native binary is several commits behind the Wasm package, so outputs may not look the same.

The Wasm package can be built using `cd rust && wasm-pack build --target web --features gpu`. Since Wasm packages are compiled as a library, the "main" function of the Wasm package can be found in `/rust/src/arch/wasm/harness.rs`.

Since the native binary is compiled as a binary, the binary entry point can be found at `/rust/bin/native.rs` and wraps `/rust/src/arch/native/harness.rs`.

If the GPU compute shaders in particle cloth or particle paper cause issues, the simulation can be reverted to the serial Gauss-Seidel CPU implementation by omitting the `gpu` feature flag. All code relating to the user interface wrapping the Wasm package is located in `/src`.

2.2 Available Simulations

URL	Simulation
/	Triangle Mesh Cloth Simulation + Rotating BVH Rigid Body
/paper	Triangle Mesh Paper Simulation
/particle_cloth	GPU Particle Cloth Simulation + Static SDF Rigid Body
/particle_paper	GPU Particle Paper Simulation

Table 1: URLs for different simulations

We offer 4 simulations located at different sub-URLs. The triangle mesh simulations run shape-matching PBD constraints by default while the particle cloth simulations only run distance XPBD constraints for cloth structure. We have a UI that allows the user to adjust several parameters relating to the simulation. Since scales for parameters $k_i \in [0, 1]$ may be very small, or on the exponential magnitude, we recommend manually entering numbers into inputs if the sliders do not prove sufficient. Damping and timestep are examples of this.

2.3 Available Crease Patterns

Crease Pattern	Description
<code>line.cp</code>	A single vertical crease
<code>lines.cp</code>	Two vertical creases
<code>waterbomb.cp</code>	A traditional origami base
<code>miura_ori.cp</code>	A classic and easy to fold origami tessellation
<code>waterbomb_tess.cp</code>	A classic and harder to fold origami tessellation
<code>yaccho.cp</code>	Modern origami "color-change" model. Designed and provided by Kei Morisue.
<code>hex_torso.cp</code>	

Table 2: Available crease patterns for paper simulation

Once you have build the necessary packages, you can run the web UI using `npm install && npm run dev`. Attached below are reference outputs you can compare to:

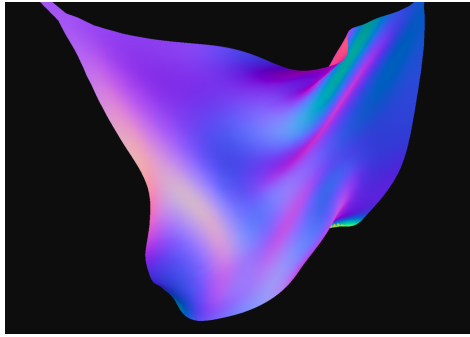


Figure 2: Triangle mesh cloth simulation: PBD shape-matching cloth at resolution=64.

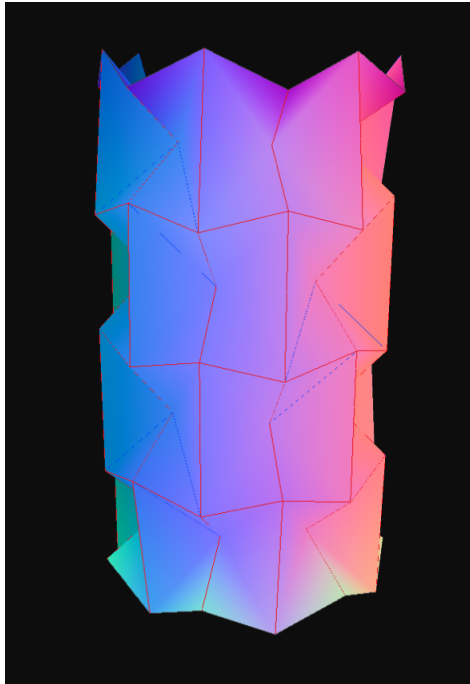


Figure 3: Triangle mesh paper simulation (XPBD shape-matching dihedral angle constraint): Waterbomb tessellation at resolution=1.

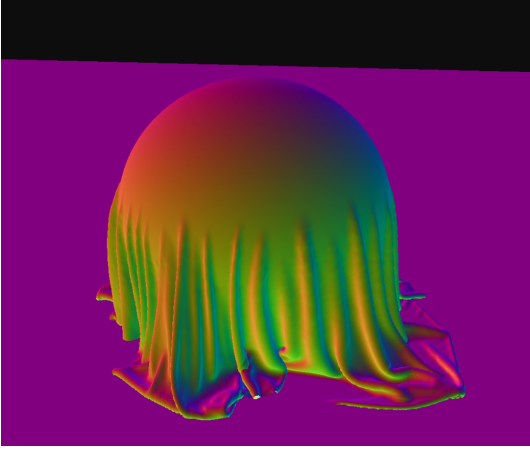


Figure 4: Particle cloth simulation (XPBD distance constraint): High-resolution cloth at 30 FPS, resolution=150, substeps = 1

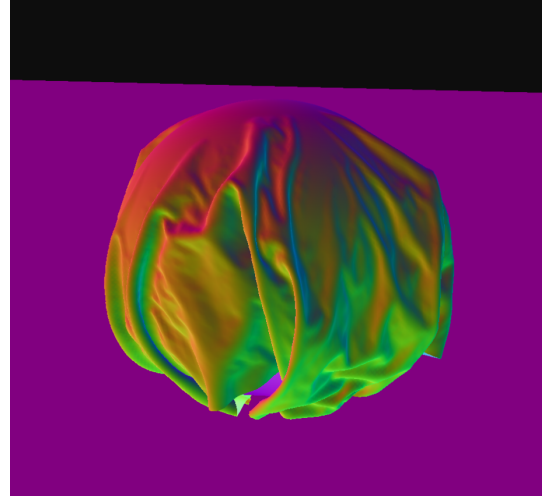


Figure 5: Particle cloth simulation (XPBD distance constraint): High-resolution cloth at 20 FPS, resolution=150, substeps = 10

From Figures 4 and 5, we see that using more substeps shows less stretching and higher material stiffness.

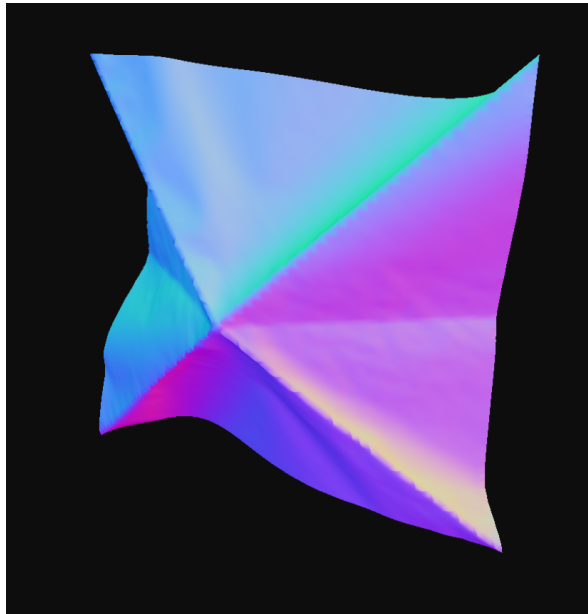


Figure 6: Particle paper simulation (XPBD distance + dihedral angle constraint): Waterbomb base in a high resolution paper, resolution = 64.

One limitation of the particle XPBD paper simulation is that in high resolutions, the distance constraints become slightly unstable. Notice the small bumps in the paper in Figure 6. When we ran simulations in the triangle mesh PBD shape-matching paper simulation, the paper was smooth. This is because shape-matching provides high-fidelity projections onto the rest shapes.

References

- [GDG18] Amanda Ghassaei, Erik D. Demaine, and Neil Gershenfeld. Fast, interactive origami simulation using GPU computation. In *Origami⁷: Proceedings of the 7th International Meeting on Origami in Science, Mathematics and Education (OSME 2018)*, volume 4, pages 1151–1166. Tarquin, Oxford, England, September 5–7 2018.

- [MMC16] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. Xpbd: Position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games*, MIG '16, pages 49–54, New York, NY, USA, 2016. ACM.
- [MSL⁺19] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. In *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA '19, New York, NY, USA, 2019. Association for Computing Machinery.